

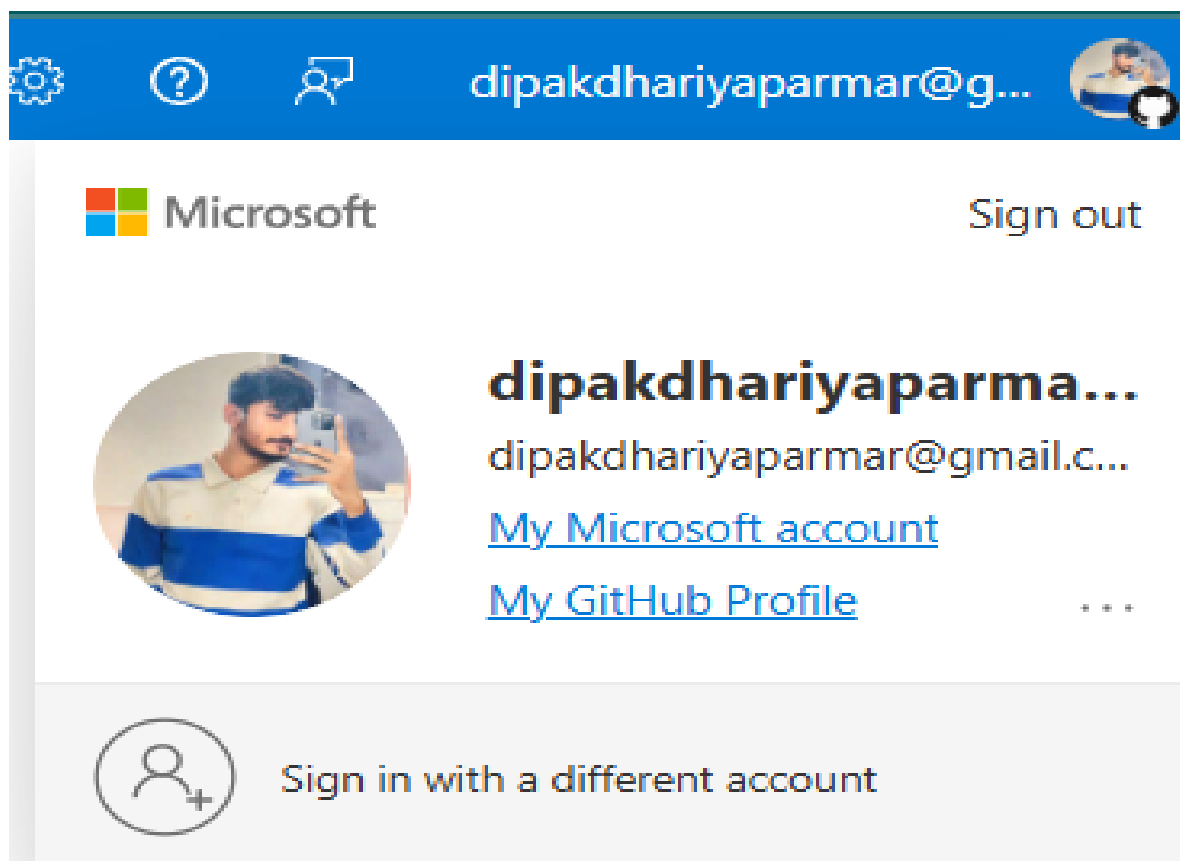
Cloud Infrastructure and Services

Practical Questions Answers

1) Setup and Configuration of a Virtual Machine (VM) on a Cloud Platform

=>

Step 1: - Log in to Azure Portal



Step 2: - Create a New Virtual Machine

=> Click on "**Virtual Machines**" in the left-hand menu.

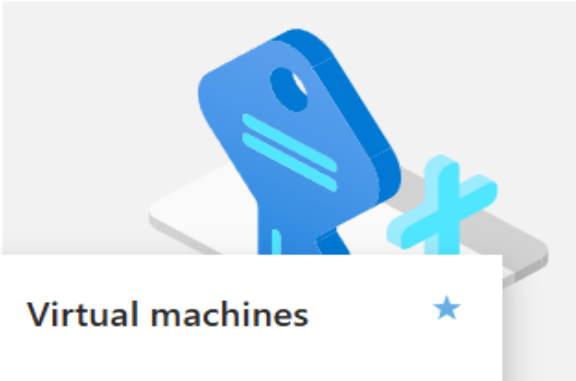
← → ↻ portal.azure.com/#home

☰

- + Create a resource
- 🏠 Home
- 📊 Dashboard
- ☰ All services
- ★ FAVORITES
- 📊 All resources
- 🗂️ Resource groups
- 🌐 App Services
- ⚡ Function App
- 🗄️ SQL databases
- 🌐 Azure Cosmos DB
- 🖥️ Virtual machines
- 🌐 Load balancers
- 📁 Storage accounts
- 🌐 Virtual networks
- 🔑 Microsoft Entra ID
- 🕒 Monitor
- 🧠 Advisor
- 🛡️ Microsoft Defender for Cloud
- 💰 Cost Management + Billing
- 👤 Help + support

Welcome to Azure!

Don't have a subscription? Check out the following:





Virtual machines

+ Create ▾ 👁 View

Description

Create a virtual machine that runs Linux or Windows. Select an image from the marketplace or use your own customized image.

Free training from Microsoft

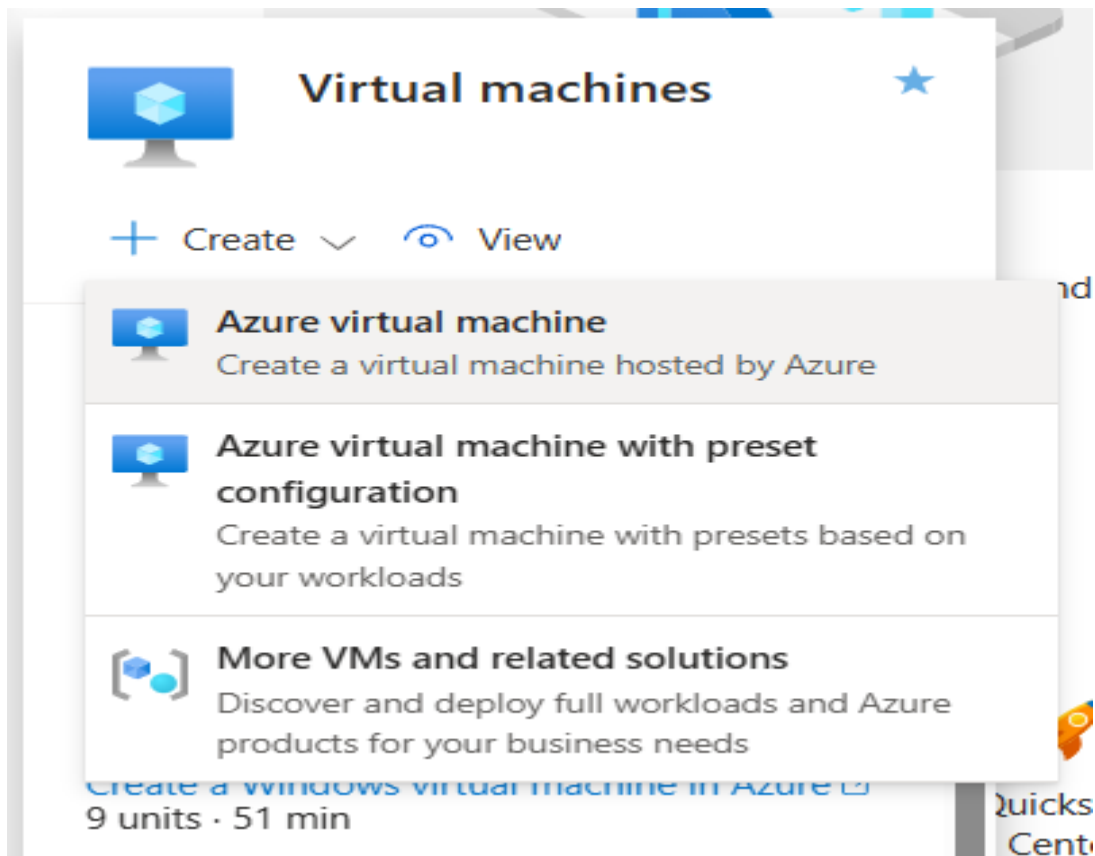
- [Introduction to Azure virtual machines](#) 8 units · 1 hr 7 min
- [Create a Windows virtual machine in Azure](#) 9 units · 51 min
- [Create a Linux virtual machine in Azure](#) 7 units · 1 hr 26 min

Useful links

- [Overview](#)
- [Get started](#)
- [Documentation](#)

Quickstart Center

=> Click on "+ Create" > "Azure virtual machine".



- => **Subscription:** Select your Azure subscription.
- => **Resource group:** Create a new one or select an existing group.
- => **Virtual machine name:** e.g., MyVM.
- => **Region:** Choose a location close to you (e.g., East US).
- => **Availability options:** Leave default.
- => **Image:** Select Windows Server 2019 or Ubuntu 20.04 LTS.
- => **Size:** Choose a VM size (e.g., Standard B1s for low-cost setup).
- => **Authentication type:** Choose Password or SSH public key.
- => **Username:** e.g., azureuser.
- => **Password:** Enter and confirm a secure password.

[Home](#) >

Create a virtual machine ...

Project details

Select the subscription to manage deployed resources and costs. Use resource groups like folders to organize and manage all your resources.

Subscription * ⓘ	
Resource group ⓘ	Loading... Create new

Instance details

Virtual machine name * ⓘ	
Region ⓘ	Loading...
Availability options ⓘ	No infrastructure redundancy required
Security type ⓘ	Trusted launch virtual machines Configure security features
Image * ⓘ	 Ubuntu Server 24.04 LTS - x64 Gen2 See all images Configure VM generation
VM architecture ⓘ	☐ Arm64 ☒ x64

[< Previous](#)

[Next : Disks >](#)

[Review + create](#)

Step 3: - Configure Network Settings

Under the **Networking tab**:

- Choose **Virtual Network** (create a new one or use default).
- Set **Public IP** to enabled to allow remote access.
- Set **Inbound ports** to Allow selected ports.
- Choose RDP (3389) for Windows or SSH (22) for Linux.

Basics Disks **Networking** Management Monitoring Advanced Tags Review + create

Define network connectivity for your virtual machine by configuring network interface card (NIC) settings. You can control ports, inbound and outbound connectivity with security group rules, or place behind an existing load balancing solution. [Learn more](#)

Network interface

When creating a virtual machine, a network interface will be created for you.

Virtual network ⓘ	Loading... Create new
Public IP ⓘ	Loading... Create new
NIC network security group ⓘ	☐ None ☒ Basic ☐ Advanced
Public inbound ports * ⓘ	☐ None ☒ Allow selected ports
Select inbound ports *	SSH (22)

< Previous

Next : Management >

Review + create

Step 4: Review and Create

1. Click **Review + Create**.
2. Wait for validation to complete.
3. Click **Create** to deploy the VM.
4. Deployment may take a few minutes.

Step 5: Access the Virtual Machine

- **Windows VM:**
 1. After deployment, go to the VM dashboard.
 2. Click on **Connect > RDP**.
 3. Download the RDP file and open it.
 4. Log in using the credentials created earlier.
- **Linux VM:**

1. Copy the **public IP address**.
2. Open a terminal and type:
`ssh azureuser@<Public-IP>`
3. Enter the password when prompted.

2) Deploy a Web Application using Infrastructure as a Service (IaaS)

=>

Step 1: Launch Windows EC2 Instance

1. Log in to [AWS Console](#)
2. Navigate to EC2 > Instances > Launch Instance
3. Fill in:
 - Name: WindowsWebServer
 - AMI: Choose Microsoft Windows Server 2022 Base
 - Instance type: t2.micro (Free Tier)
 - Key Pair: Select/Create one (required for RDP access)
 - Security Group:
 - Allow RDP (port 3389) – for remote access
 - Allow HTTP (port 80) – for web server
4. Click Launch Instance

Name and tags [Info](#)

Name

WindowsWebServer

[Add additional tags](#)

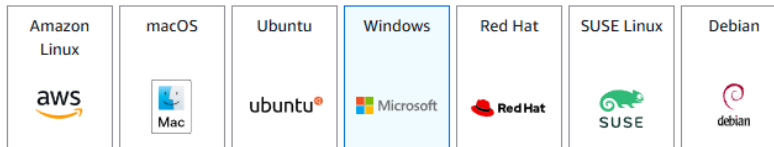
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start



[Browse more AMIs](#)

Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

Microsoft Windows Server 2025 Base
ami-0fd8b8b04c5329912 (64-bit (x86))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

Description

Microsoft Windows 2025 Datacenter edition. [English]

Microsoft Windows Server 2025 Full Locale English AMI provided by Amazon

Architecture	AMI ID	Publish Date	Username
64-bit (x86)	ami-0fd8b8b04c5329912	2025-05-15	Administrator

Verified provider

Step 2: Connect to Instance via RDP

1. Wait for instance to be in **Running** state.
2. Select the instance and click **Connect** → **RDP Client**.
3. Click **Get Password** (after ~4 minutes)
 - Upload your .pem key to decrypt the Windows password.
4. Use **Remote Desktop Connection (RDP)** on your computer:
 - Host: Public IP
 - Username: Administrator
 - Password: (decrypted password)

▼ Instance type

Info | Get advice

Instance type

t3.micro

Free tier eligible

Family: t3 2 vCPU 1 GiB Memory Current generation: true

On-Demand Ubuntu Pro base pricing: 0.0143 USD per Hour On-Demand RHEL base pricing: 0.0396 USD per Hour

On-Demand SUSE base pricing: 0.0108 USD per Hour On-Demand Linux base pricing: 0.0108 USD per Hour

On-Demand Windows base pricing: 0.02 USD per Hour

All generations

Compare instance types

Additional costs apply for AMIs with pre-installed software

▼ Key pair (login)

Info

You can use a key pair to securely connect to your instance. Ensure that you have access to the selected key pair before you launch the instance.

Key pair name - required

WindowsWebServerkey

▼

Create new key pair

For Windows instances, you use a key pair to decrypt the administrator password. You then use the decrypted password to connect to your instance.

▼ Network settings

Info

Edit

Network | Info

vpc-0b0dffc440d6d9a88

Subnet | Info

No preference (Default subnet in any availability zone)

Auto-assign public IP | Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups) | Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

We'll create a new security group called 'launch-wizard-4' with the following rules:

Allow RDP traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Step 3: Connect to the Instance

- For **Windows AMI**:
 - Go to **Instances** → **Select your instance** → **Connect** → **RDP Client**
 - Download the RDP file and decrypt the password
 - Open the RDP file and log in using the decrypted credentials

Step 4: Install Web Server

- **For Windows (IIS):**
 1. Open **Server Manager**
 2. Click **Add roles and features**
 3. Select **Web Server (IIS)** → Install
 4. Go to: C:\inetpub\wwwroot
 5. Place your web application files here (e.g., index.html, default.aspx)

Step 5: Test the Web Application

- Open a browser on your local machine
- Enter the **Public IPv4 address** of your EC2 instance:
http://<public-ip>

3) Create and Manage Storage Buckets on Cloud Storage

=>

Step 1: Open S3 Service

1. In the AWS Console, search for **S3** in the search bar at the top.
2. Click on **S3** under “Storage”.

Step 2: Create a Storage Bucket

1. Click **Create bucket**.
2. Fill out the **Bucket name** (must be globally unique, e.g., my-unique-bucket-name-2025).
3. Choose the **AWS Region** closest to your users or workload.
4. Configure the following options:

- **Block Public Access:** Keep enabled unless you need public access.
- **Bucket Versioning:** Enable if you want to keep versions of files.
- **Encryption:** Optional, but recommended for security.
- Leave other settings as default for now.

5. Click **Create bucket** at the bottom.

Step 3: Upload Files to the Bucket

1. Click the **bucket name** to open it.
2. Click **Upload** → **Add files** (select a file like index.html, image.jpg, etc.).
3. Click **Upload**.

Step 4: Manage Bucket Objects

- You can:
 - View uploaded files
 - Delete files
 - Download files
 - Change permissions (e.g., make public)

Step 5: Make a File Public

1. Click on the file (object).
2. Go to **Permissions** tab.
3. Under **Object Ownership**, make sure ACLs are enabled.
4. Scroll to **Access Control List (ACL)** → click **Edit**.
5. Under **Public access**, check **Read** permission for “Everyone”.
6. Save changes.

4) Monitor Cloud Infrastructure using Cloud Monitoring Tools

=>

Step 1: Launch a Cloud Resource

Launch an AWS resource such as an EC2 instance, RDS database, or S3 bucket.

Assume i already have an EC2 instance running

Step 2: Open CloudWatch

1. Sign in to the **AWS Management Console**.
2. Search for **CloudWatch** in the search bar and open it.

Step 3: View Default Metrics

1. Go to **Metrics** from the left sidebar.
2. Choose **EC2 > Per-Instance Metrics**.
3. Select your instance ID.
4. View metrics like:
 - CPUUtilization
 - DiskReadBytes
 - NetworkIn/Out
 - StatusCheckFailed

Step 4: Create a CloudWatch Alarm

1. In CloudWatch, click on **Alarms > Create Alarm**.
2. Select a metric (e.g., CPUUtilization).
3. Set a threshold (e.g., **if CPU > 80% for 5 minutes**).
4. Choose notification options (e.g., send email via SNS).
5. Name the alarm and create it.

Step 5: Enable Detailed Monitoring (Optional)

By default, EC2 sends metrics every 5 minutes.

To enable **1-minute intervals**:

1. Go to **EC2 Dashboard**.
2. Select your instance → **Actions > Monitor and troubleshoot > Enable detailed monitoring**.

Step 6: View Logs (Optional)

You can send logs to CloudWatch from:

EC2 (via CloudWatch agent)

Lambda

ECS

Applications

5) Implement Serverless Computing using AWS Lambda or Google Cloud Functions

=>

Step 1: Open AWS Lambda Console

1. Go to <https://console.aws.amazon.com/lambda>
2. Click **Create function**.

Step 2: Configure Function

- **Function name:** HelloLambda
- **Runtime:** Node.js 18.x (or Python 3.9, etc.)
- **Permissions:**
Choose *Create a new role with basic Lambda permissions*.

- Click **Create function**.

Step 3: Add Function Code

In the **Code** section, replace existing code with:

```
exports.handler = async (event) => {  
  return {  
    statusCode: 200,  
    body: JSON.stringify({ message: "Hello from Lambda!" }),  
  };  
};
```

=>Click **Deploy** to save changes.

Step 4: Add an API Gateway Trigger

1. Click **Add trigger**.
2. Choose **API Gateway**.
3. Select:
 - API type: HTTP API
 - Security: Open (or choose auth if needed)
4. Click **Add**.

Step 5: Test the Function

- Copy the **API endpoint URL** shown in the trigger.
- Open it in a browser or use Postman/cURL:

```
curl https://lambdaexample.execute-api.region.amazonaws.com/
```

6) Design and Deploy a Database using Cloud Database Services

=>

Step 1: Log in to AWS Console

Go to <https://console.aws.amazon.com/> and search for **RDS** in the AWS services.

Step 2: Create a New Database

1. Click **Create Database**.
2. **Choose a database creation method:**
 - Select **Standard create**.
3. **Engine options:**
 - Choose **MySQL**.
 - Version: e.g., MySQL 8.0.34
4. **Templates:**
 - Select **Free tier** (for practice/testing)

Step 3: Configure Database Settings

- **DB instance identifier:** mydbinstance
- **Master username:** admin
- **Password:** set and confirm a secure password

Step 4: DB Instance Configuration

- **DB instance class:** db.t3.micro (free tier)
- **Storage:** 20 GiB (default is fine)

Step 5: Connectivity

- **Virtual Private Cloud (VPC):** Default
- **Public access:** Select **Yes** (to allow access from outside)

- **VPC security group:** Create new or select default

Step 6: Create Database

Click **Create database**. It may take a few minutes to launch.

Step 7: Connect to the Database

1. Go to the **Databases** section and click your DB name.
2. Copy the **Endpoint** (e.g., mydbinstance.xyz.amazonaws.com)
3. Open **MySQL Workbench** or use a terminal:
 - **Host:** <your-endpoint>
 - **Port:** 3306
 - **Username:** admin
 - **Password:** Your password

7) Setting Up an S3 Bucket and Uploading Files

=>

Step 1: Log in to AWS Console

1. Visit <https://console.aws.amazon.com/>.
2. In the search bar, type **S3** and select **Amazon S3**.

Step 2: Create an S3 Bucket

1. Click **Create bucket**.
2. Enter a **unique bucket name** (e.g., my-web-bucket-2025).
3. Choose your **AWS Region**.

4. Keep default settings (for now).
5. Uncheck **Block all public access** if you plan to host public content.
6. Acknowledge the warning and click **Create bucket**.

Step 3: Upload Files

1. Click the name of your bucket to open it.
2. Click **Upload > Add files**.
3. Choose files like:
 - index.html
 - image.jpg
 - style.css
4. Click **Upload**.

Example Dummy Files to Upload

1. index.html:

```
<!DOCTYPE html>

<html>

<head>

  <title>My S3 Website</title>

</head>

<body>

  <h1>Welcome to My Website!</h1>

</body>

</html>
```


2. image.jpg:

Upload a dummy examples image

3. style.css:

```
body {  
    background-color: #f2f2f2;  
    font-family: Arial, sans-serif;  
}  
  
h1 {  
    color: #333;  
}
```

Step 4: Make Files Public (For Website Hosting)

1. Select a file (like index.html).
2. Click **Actions** > **Make public** (you may need to configure a bucket policy first).

Step 5 (Optional): Enable Static Website Hosting

1. Go to your bucket → **Properties** tab.
2. Scroll down to **Static website hosting**.
3. Enable it and set:
 - **Index document:** index.html
4. Save changes.
5. Use the generated URL to view your hosted file.

8) Storing and Retrieving Data in DynamoDB

=>

Step 1: Open DynamoDB

1. Go to the **AWS Console**.
2. In the search bar, type **DynamoDB** and open the service.

Step 2: Create a DynamoDB Table

1. Click **“Create table”**.
2. Enter:
 - **Table name:** Students
 - **Partition key:** StudentID (Type: String or Number)
 - Leave Sort Key empty (for simple table)
3. Leave default settings or adjust as needed (e.g., on-demand capacity).
4. Click **“Create table”**.

Step 3: Insert (Store) Data

1. After the table is created, open it.
2. Go to the **Items** tab.
3. Click **“Create item”**.
4. In JSON editor mode, enter something like:

```
{  
  "StudentID": "101",  
  "Name": "Dipak Dhariyaparmar",  
  "Course": "Cloud Computing",  
  "Grade": "A"
```

}

5. Click **“Create item”** to save.

Step 4: Retrieve (Read) Data

1. On the **Items** tab, use the **search/filter box** to find items by key.
2. Example: Enter StudentID = 101.
3. Click the result to view full item details.

9) Creating an IAM User and Assigning S3 Access Permissions

=>

Step 1: Open the IAM Console

1. Go to the [IAM Console](#)
2. In the left panel, click on **Users**.
3. Click **Add users**.

Step 2: Create the IAM User

1. **User name:** e.g., s3user
2. **Access type:**
 - Check **Programmatic access** (for CLI/SDK)
 - Check **AWS Management Console access** (for UI access)
3. Set a **password** if enabling console access.
4. Click **Next: Permissions**.

Step 3: Attach Policies (S3 Access)

You have two main options:

Option A: Full Access to All S3 Buckets

1. Choose **Attach existing policies directly**
2. Search for and select:
 - AmazonS3FullAccess

Option B: Limited Access to One Bucket (Custom Policy)

1. Choose **Attach policies directly**
2. Click **Create policy** → Go to **JSON** tab
3. Paste the following policy (replace your-bucket-name):

```
{
  "Version": "2012-10-17",
  "Statement": [
    {
      "Effect": "Allow",
      "Action": ["s3:ListBucket"],
      "Resource": ["arn:aws:s3:::your-bucket-name"]
    },
    {
      "Effect": "Allow",
      "Action": [
        "s3:GetObject",
        "s3:PutObject",
        "s3:DeleteObject"
      ]
    }
  ]
}
```

```
    "Resource": ["arn:aws:s3:::your-bucket-name/*"]
  }
]
}
```

4. Click **Next: Tags** (optional) → **Next: Review**

5. Click **Create policy**, then go back and attach it to your user.

Step 4: Review and Create

1. Review the details
2. Click **Create user**

Step 5: Verify Permissions

log in as the new IAM user (if console access is enabled) or configure AWS CLI with:

aws configure

or

aws s3 ls

and then upload a file

aws s3 cp myfile.txt s3://your-bucket-name/

10) Connecting to an EC2 Instance Using SSH

=>

Step 1: Locate Your .pem File

This is the private key file downloaded when you launched your EC2 instance.

Example: my-key.pem

Move it to a secure location and open a terminal in that directory.

Step 2: Change Permission of .pem File using gitbash

```
chmod 400 my-key.pem
```

Step 3: Find Your Public IP

1. Go to AWS Console → EC2 Dashboard.
2. Click on **Instances** → Select your instance.
3. Copy the **Public IPv4 address**.

Step 4: Connect to EC2 via SSH

If your instance is Amazon Linux or Ubuntu, use the appropriate default username:

- **Amazon Linux 2:** ec2-user
- **Ubuntu:** ubuntu
- **CentOS:** centos
- **RHEL:** ec2-user

```
ssh -i my-key.pem ec2-user@your-public-ip
```

example: -

```
ssh -i my-key.pem ubuntu@54.202.45.123
```

11) Hosting a Static Website on AWS S3

=>

Step 1: Create an S3 Bucket

1. Go to the **AWS Management Console**.
2. Navigate to **S3**.
3. Click **Create bucket**.

4. Enter a **unique bucket name** (e.g., my-static-website-123).
5. Choose a region (e.g., us-east-1).
6. **Uncheck** "Block all public access".
7. Acknowledge the warning checkbox.
8. Click **Create bucket**.

Step 2: Upload Website Files

1. Open your created bucket.
2. Click **Upload** → **Add files**.
3. Select your website files (e.g., index.html, style.css, image.jpg).
4. Click **Upload**.

Step 3: Set Bucket Policy for Public Access

1. Go to your bucket → **Permissions** tab.
2. Scroll to **Bucket Policy** → Click **Edit**.
3. Paste the following (replace your-bucket-name with your actual bucket name):

```
{  
  "Version": "2012-10-17",  
  "Statement": [  
    {  
      "Sid": "PublicReadGetObject",  
      "Effect": "Allow",  
      "Principal": "*",  
      "Action": "s3:GetObject",
```

```
"Resource": "arn:aws:s3:::your-bucket-name/*"
}
]
}
```

4. Click **Save changes**.

Step 4: Enable Static Website Hosting

1. Go to the **Properties** tab of the bucket.
2. Scroll to **Static website hosting** → Click **Edit**.
3. Choose **Enable**.
4. For **Index document**, enter: index.html.
 - (Optional) For **Error document**, enter: error.html.
5. Click **Save changes**.

Step 5: Access Your Website

1. In the **Properties** tab under **Static website hosting**, you'll see a **Bucket website endpoint**.
2. Open that URL in your browser — your static website is now live!

Example File: index.html

```
<!DOCTYPE html>

<html>

<head>

<title>My Static Website</title>
```



```
<style>

  body { font-family: Arial; background: #f4f4f4; text-align: center; padding: 50px;
}

  h1 { color: #333; }

</style>

</head>

<body>

  <h1>Welcome to My Static Website!</h1>

  <p>This site is hosted on AWS S3.</p>

</body>

</html>
```

12) Creating and Managing a DynamoDB Table with Primary Keys

=>

Step 1: Open DynamoDB in AWS Console

1. Log in to your [AWS Management Console](#).
2. Navigate to **DynamoDB** service.

Step 2: Create a New Table

1. Click on **“Create table”**.
2. Fill in the following details:
 - **Table name:** e.g., Students
 - **Partition key (Primary Key):** e.g., StudentID (Type: String or Number)

- *(Optional)* **Sort key**: e.g., CourseID (Type: String or Number) — if you want a composite key.
- 3. Leave the default **Capacity mode** as “**On-demand**” (you can choose provisioned if you want to manually manage read/write capacity).
- 4. Click **Create table**.

Step 3: Add Data (Items) to the Table

1. After the table is created, go to the “**Items**” tab.
2. Click **Create item**.
3. Add values for:
 - StudentID: e.g., 101
 - CourseID: e.g., CS101 *(if using sort key)*
 - Other attributes: e.g., Name, Email, Grade
4. Click **Save**.

Step 4: View or Edit Items

- In the **Items** tab, you can:
 - View, edit, or delete items.
 - Use “**Scan**” or “**Query**” to search items based on keys.

Step 5: Monitor and Manage Table

- Use the “**Metrics**” tab to monitor usage and performance.
- Use the “**Indexes**” tab to create **Global Secondary Indexes (GSI)** for more flexible querying.
- Use “**Backups**” to enable point-in-time recovery.

Example Table Schema: Students

StudentID (Partition Key)	CourseID (Sort Key)	Name	Grade
101	CS101	Dipak	A+
102	CS101	Ayushi	A

13) Launch and Connect to an EC2 Instance (Linux)

=>

Step 1: Log in to AWS Console

- Go to: <https://console.aws.amazon.com>
- Search for **EC2** and click on it.

Step 2: Launch a New Instance

1. Click “**Launch Instance**” on the EC2 Dashboard.
2. Fill in:
 - **Name:** e.g., DipakLinuxServer
 - **Amazon Machine Image (AMI):** Select **Amazon Linux 2023 AMI (HVM), SSD Volume Type** (*free tier eligible*).
 - **Instance type:** Choose **t2.micro** (*Free Tier eligible*).
 - **Key pair:** Choose existing or create new key pair.
 - If creating new, download the .pem file and **save it safely**.
 - **Network settings:** Allow **SSH (port 22)** from **My IP** or **Anywhere (0.0.0.0/0)** if needed.
3. Leave defaults for storage and other settings.
4. Click **Launch Instance**.

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

DipakLinuxServer

Add additional tags

▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Q Search our full catalog including 1000s of application and OS images

Recents

Quick Start

Amazon Linux



macOS



Ubuntu



Windows



Red Hat



SUSE Linux



Debian





Browse more AMIs

Including AMIs from AWS, Marketplace and the Community

Amazon Machine Image (AMI)

Amazon Linux 2023 AMI

ami-00f34bf9aeacdf007 (64-bit (x86), uefi-preferred) / ami-0430bc22c15775e7f (64-bit (Arm), uefi)

Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

Description

Amazon Linux 2023 is a modern, general purpose Linux-based OS that comes with 5 years of long term support. It is optimized for AWS and designed to provide a secure, stable and high-performance execution environment to develop and run your cloud applications.

Amazon Linux 2023 AMI 2023.7.20250512.0 x86_64 HVM kernel-6.1

Architecture

64-bit (x86) ▼

Boot mode

uefi-preferred

AMI ID

ami-00f34bf9aeacdf007

Publish Date

2025-05-09

Username ⓘ

ec2-user

Verified provider

Create key pair



Key pair name

Key pairs allow you to connect to your instance securely.

DipakLinuxServerKey

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

 When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#) 

Cancel

Create key pair

▼ Network settings [Info](#)

[Edit](#)

Network [Info](#)

vpc-0b0dff440d6d9a88

Subnet [Info](#)

No preference (Default subnet in any availability zone)

Auto-assign public IP [Info](#)

Enable

[Additional charges apply](#) when outside of [free tier allowance](#)

Firewall (security groups) [Info](#)

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

☒ Create security group

☐ Select existing security group

We'll create a new security group called 'launch-wizard-6' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere

0.0.0.0/0

☐ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☐ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

14) Launch and Connect to an EC2 Instance (Windows)

=>

Step 1: Sign in to AWS Console

- Go to <https://console.aws.amazon.com>
- From the **Services** menu, choose **EC2**.

Step 2: Launch a New Instance

1. Click **“Launch instance”**.
2. Fill the details:
 - **Name:** Example – DipakWindowsServer22
 - **Amazon Machine Image (AMI):** Choose **Microsoft Windows Server 2022 Base** (or 2019/2016) (*Free tier eligible*)
 - **Instance Type:** Select **t2.micro** (*Free Tier eligible*).
 - **Key Pair (login):**
 - Select an existing key pair or
 - Click **Create new key pair** → Choose .pem format → Download it and save securely.
 - **Network Settings:**
 - Allow **RDP (port 3389)** from **My IP** or **Anywhere** (0.0.0.0/0) if needed for testing.
3. Leave storage and advanced settings as default.
4. Click **Launch Instance**.

Launch an instance [Info](#)

Amazon EC2 allows you to create virtual machines, or instances, that run on the AWS Cloud. Quickly get started by following the simple steps below.

Name and tags [Info](#)

Name

DipakWindowsServer22

[Add additional tags](#)

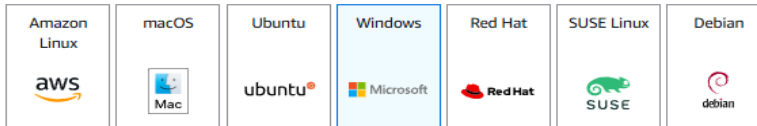
▼ Application and OS Images (Amazon Machine Image) [Info](#)

An AMI is a template that contains the software configuration (operating system, application server, and applications) required to launch your instance. Search or Browse for AMIs if you don't see what you are looking for below

Search our full catalog including 1000s of application and OS images

Recents

Quick Start



[Browse more AMIs](#)
Including AMIs from
AWS, Marketplace and
the Community

Amazon Machine Image (AMI)

Microsoft Windows Server 2025 Base
ami-0fd8b8b04c5329912 (64-bit (x86))
Virtualization: hvm ENA enabled: true Root device type: ebs

Free tier eligible ▼

Description

Microsoft Windows 2025 Datacenter edition. [English]

Microsoft Windows Server 2025 Full Locale English AMI provided by Amazon

Architecture

64-bit (x86)

AMI ID

ami-0fd8b8b04c5329912

Publish Date

2025-05-15

Username

Administrator

Verified provider

Create key pair



Key pair name

Key pairs allow you to connect to your instance securely.

DipakWindowsServer22

The name can include up to 255 ASCII characters. It can't include leading or trailing spaces.

Key pair type

☒ **RSA**
RSA encrypted private and public key pair

☐ **ED25519**
ED25519 encrypted private and public key pair (Not supported for Windows instances)

Private key file format

☒ **.pem**
For use with OpenSSH

☐ **.ppk**
For use with PuTTY

When prompted, store the private key in a secure and accessible location on your computer. **You will need it later to connect to your instance.** [Learn more](#)

Cancel

Create key pair

▼ Network settings

Info

Edit

Network

Info

vpc-0b0dff440d6d9a88

Subnet

Info

No preference (Default subnet in any availability zone)

Auto-assign public IP

Info

Enable

Additional charges apply when outside of free tier allowance

Firewall (security groups)

Info

A security group is a set of firewall rules that control the traffic for your instance. Add rules to allow specific traffic to reach your instance.

Create security group

Select existing security group

We'll create a new security group called 'launch-wizard-2' with the following rules:

☒ Allow SSH traffic from

Helps you connect to your instance

Anywhere
0.0.0.0/0

☐ Allow HTTPS traffic from the internet

To set up an endpoint, for example when creating a web server

☐ Allow HTTP traffic from the internet

To set up an endpoint, for example when creating a web server

Get Windows password

Info

Use your private key to retrieve and decrypt the initial Windows administrator password for this instance.

Instance ID

i-060f40bccd50093da (DipakWindowsServer)

Key pair associated with this instance

Dipakwindowkey

Private key

Either upload your private key file or copy and paste its contents into the field below.

Upload private key file

Dipakwindowkey.pem

1.678KB

Private key contents - optional

```
-----BEGIN RSA PRIVATE KEY-----
MIIEpAIBAAKCAQEAu8DKNwv2FvVYWWtdOCPcyMKXFr7jvdtZctC7FPoXwnkeBY
aKlgO7JXGpHokMV1VDua5g9ZfRf8OC8XAkNYVYsdytlg6i/RHkJVfHabyviOu
l5sE++Tdl1LyWlH+SPdtbbbs9l3Ywlg3GUmREG/EkpDvQxt8vVil+A0BNsgfrhtd
5nSxvF1Uuu+BZtAYntCocYp5+YuihijAYa517G6HibJg/UBXRYtAtp02piB1wMH6
WfaAv7wQYATxOAgpGKm3LCKK4mkW7MHYMODlIBN51o8zfEneOwwYzfZXnKW52BNJ
WXAK3Z3yWk4/9t+Ttsg46LNyuh7BIUQVe1MQIDAQABAgIa+Ut+RpcCHIUU
7hCC9ekQwvwwX/rt3E2SpDI+/vn3DTqV3G1s6n6Wl+e47P2gcqFNd8gW6XMXbGb
-----
```

Cancel

Decrypt password

Step 3: Wait for the Instance to Launch

- Click “**View all instances**”.
- Wait until the **Instance State** becomes **Running**.
- Confirm **Status Check** shows **2/2 checks passed**.

Step 4: Connect to the Windows Instance via RDP

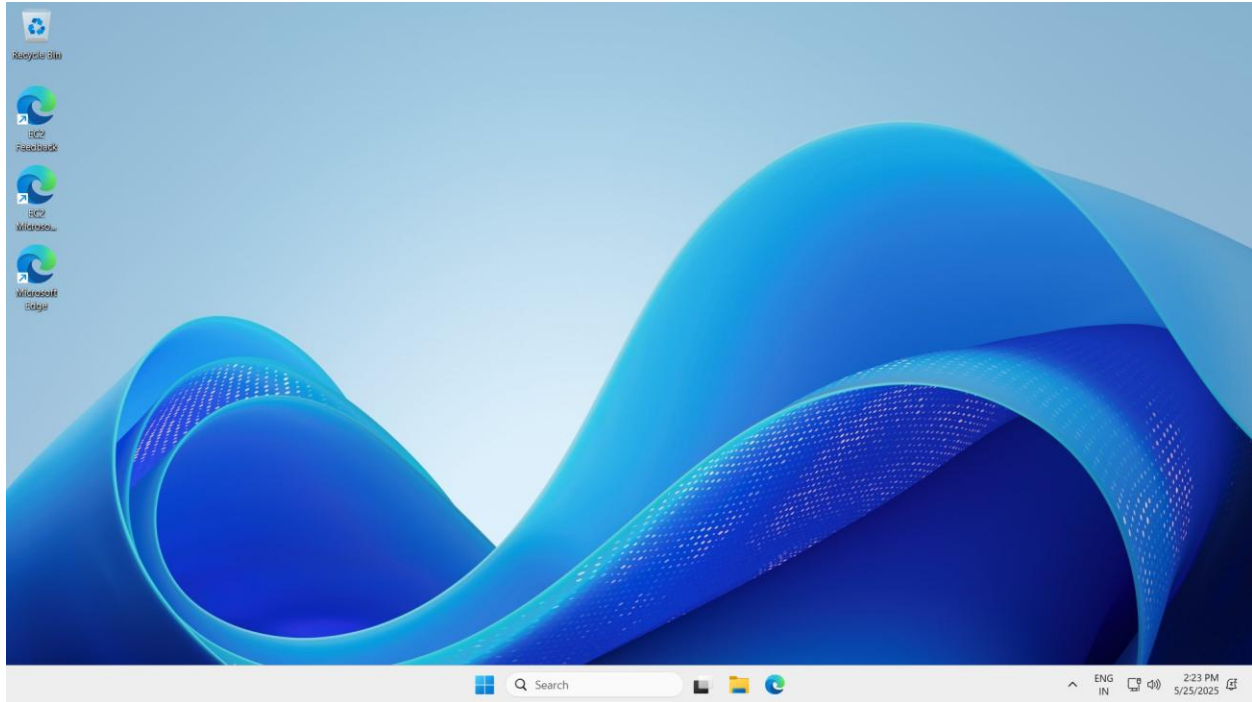
- Select the instance.

2. Click **Connect** (top right).
3. In the Connect window:
 - Select **RDP client** tab.
 - Click **Get Password**.
 - It may take a few minutes after launch to be ready.
 - Upload your .pem key file.
 - Click **Decrypt Password**.
4. Copy the **username** (usually Administrator) and **decrypted password**.

Step 5: Open Remote Desktop Connection (RDP)

On Windows:

1. Open **Remote Desktop Connection** (Press Win + R, type mstsc, press Enter).
2. In **Computer** field, paste the **Public IPv4 address** from the instance.
3. Click **Connect**.
4. Enter:
 - **Username:** Administrator
 - **Password:** The one you decrypted earlier
5. Accept any certificate warning and click **Yes**.



15) Upload and Delete Objects from an S3 Bucket

=>

Step 1: Sign in to AWS Management Console

- Go to <https://console.aws.amazon.com/>
- Navigate to **S3** under the "Services" menu.

Step 2: Open Your S3 Bucket

1. Click on the name of the S3 bucket you want to work with.
2. This opens the **bucket details page**.

Step 3: Upload Files to the S3 Bucket

1. Click on the **“Upload”** button.
2. In the **Upload** screen:
 - Click **“Add files”** and select files like:

- index.html (for static site)
 - .jpg, .png, .pdf, .txt etc.
 - You can also add folders (optional).
3. Click **Next** → Keep default permissions or change as needed.
 4. Click **Next** → Keep default storage class (Standard).
 5. Click **Upload**.

Step 4: Verify the Uploaded Files

- Once the upload is complete, you'll see the files listed in the **bucket's object list**.

Step 5: Delete Files from the S3 Bucket

1. In the object list, **select the checkbox** next to the file(s) you want to delete.
2. Click the **“Delete”** button at the top.
3. A confirmation window appears. Click **Delete objects**.

16) Host a Static Website Using S3

=>

Step 1: Sign in to AWS Management Console

1. Go to <https://console.aws.amazon.com/>
2. Search for **S3** in the services menu and open it.

Step 2: Create an S3 Bucket

1. Click **“Create bucket”**.
2. Enter a **unique bucket name** (e.g., my-static-site-123).
3. Choose a region.

4. **Uncheck** “Block all public access” under *Bucket settings for Block Public Access*.

Static website files must be publicly accessible.

5. Acknowledge the warning and click **Create bucket**.

Step 3: Upload Your Website Files

1. Open the bucket you just created.
2. Click **“Upload”** → **“Add files”** and select:
 - index.html (home page)
 - error.html (error page, optional)
 - Images or CSS files if needed
3. Click **Upload**.

Example of dummy file

.....

index.html

....

<html>

<head>

<title>My Static Site</title>

</head>

<body>

<h1>Welcome to My Static Website on AWS S3!</h1>

<p>This site is hosted using Amazon S3.</p>

</body>

```
</html>
```

error.html

...

```
<html>
```

```
<head>
```

```
  <title>Error</title>
```

```
</head>
```

```
<body>
```

```
  <h1>Oops! Page not found.</h1>
```

```
</body>
```

```
</html>
```

Step 4: Make Files Public

1. Select all uploaded files.
2. Click **Actions → Make public**.
3. Confirm the prompt.

Step 5: Enable Static Website Hosting

1. Go to the **“Properties”** tab of your bucket.
2. Scroll down to **“Static website hosting”**.
3. Click **Edit** and:
 - Choose **“Enable”**.
 - Specify index.html for **Index document**.

- Optionally set error.html for **Error document**.
4. Click **Save changes**.

Step 6: Access Your Website

- Copy the **website endpoint URL** and paste it in your browser.
- You should see your index.html page loaded.

17) Set Up Bucket Versioning and Object Restoration

=>

Step 1: Sign in to AWS Management Console

1. Visit <https://console.aws.amazon.com/>.
2. Search for and open the **S3** service.

Step 2: Create or Choose an S3 Bucket

1. Either **create a new bucket** or **choose an existing bucket**.
2. For a new bucket, give it a **unique name**, select your **region**, and proceed with default settings or configure as needed.

Step 3: Enable Bucket Versioning

1. Open your selected bucket.
2. Go to the **Properties** tab.
3. Scroll to **Bucket Versioning**.
4. Click **Edit**, then:
 - Select **Enable**.
 - Click **Save changes**.

Step 4: Upload and Modify a File to Generate Versions

1. Go to the **Objects** tab.
2. Click **Upload** → **Add files** → select a file like index.html.
3. Click **Upload**.

To create a new version:

- Upload the **same file** again (with changes if needed).
- AWS will **store it as a new version**, not overwrite the old one.

Step 5: View Object Versions

1. In the **Objects** tab, click “**Show versions**” (top-right).
2. You will now see:
 - Latest version
 - Older versions with version IDs

Step 6: Restore (Download or Revert to) a Previous Version

To **restore or revert**:

- Select an **older version** from the list.
- You can either:
 - **Download** it to save locally.
 - **Copy** and re-upload it to make it the latest version.

To **delete a version**:

- Select the specific version and choose **Delete**.

18) Enable Server-Side Encryption for S3 Objects

=>**Server-Side Encryption (SSE)** protects your data at rest by automatically encrypting data when it is written to Amazon S3 and decrypting it when accessed.

S3 offers three SSE options:

- **SSE-S3** (default Amazon S3-managed keys)
- **SSE-KMS** (AWS Key Management Service-managed keys)
- **SSE-C** (Customer-provided keys – less common)

Step 1: Sign in to AWS Console

1. Go to <https://console.aws.amazon.com/s3/>.
2. Click **Buckets** and choose or create a bucket.

Step 2: Enable Default Encryption for the Bucket (Optional but Recommended)

1. Select your bucket → Go to **Properties**.
2. Scroll to **Default encryption**.
3. Click **Edit**.

Choose one of the encryption options:

- **SSE-S3** (easiest option):
 - Select "**Enable**", choose **SSE-S3**.
 - **SSE-KMS**:
 - Select **SSE-KMS**, choose a **KMS key** (AWS managed or customer managed).
4. Click **Save changes**.

Step 3: Manually Encrypt an Object on Upload (Optional)

When uploading individual files, you can choose to encrypt them manually:

1. In the **S3 bucket**, click **Upload** → **Add files**.

2. After choosing files, click **Next** until you reach "**Properties**".
3. Under "**Server-side encryption**", choose:
 - **Amazon S3 key (SSE-S3)**
 - **AWS KMS key (SSE-KMS)** and pick a key
4. Complete the upload.

Step 4: Verify Encryption

To check encryption status:

1. Go to the bucket → **Objects**.
2. Click on an object name.
3. Under "**Properties**", look for **Server-side encryption**.
4. You'll see either:
 - SSE-S3
 - SSE-KMS with key ID

19) Set Up S3 Lifecycle Rules for Object Expiration

=>**Lifecycle rules** help you **automate** the **transition** of objects to different storage classes (e.g., Glacier) or **expire (delete)** objects after a defined period.

Step 1: Sign in to AWS S3 Console

1. Go to <https://console.aws.amazon.com/s3/>.
2. Click on **Buckets**, then select the bucket you want to apply the rule to.

Step 2: Go to Management Tab

1. In the bucket dashboard, click on the **Management** tab.
2. Under "**Lifecycle rules**", click **Create lifecycle rule**.

Step 3: Configure Rule Name and Scope

1. **Rule name:** Enter a name (e.g., DeleteAfter30Days).
2. **Choose scope:**
 - **Apply to all objects** — applies to everything.
 - Or choose **prefix** or **tags** — to target specific folders/files.

Step 4: Add Lifecycle Rule Action

1. **Choose action:** Select **Expire current versions of objects**.
2. **Days after object creation:** Enter **30** (or your desired number).
3. (Optional) You can also:
 - **Transition to other storage classes** before expiration.
 - **Expire previous versions** of versioned objects.

Step 5: Review and Create Rule

1. Click **Next**, then review the summary.
2. Click **Create rule**.

20) Create a Simple Lambda Function to Log Events

=>

Step 1: Sign in to AWS Management Console

1. Go to: <https://console.aws.amazon.com/lambda/>
2. Click **Create function**

Step 2: Configure the Function

- **Function name:** LogEventFunction

- **Runtime:** Choose either:
 - **Node.js 18.x**
 - **Python 3.12**
- **Permissions:** Choose “Create a new role with basic Lambda permissions”

Click **Create function**

Step 3: Add Code to the Function

In Node.js

```
exports.handler = async (event) => {  
  console.log("Event Received:", JSON.stringify(event, null, 2));  
  return {  
    statusCode: 200,  
    body: JSON.stringify('Event logged successfully!'),  
  };  
};
```

=>Click **Deploy** after adding your code.

Step 4: Test the Function

1. Click **Test**
2. Create a new test event:
 - **Event name:** TestEvent
 - Keep the default JSON (or customize it)

```
{  
  "name": "Dipak Dhariyaparmar",  
  "action": "login"
```

}

3. Click **Test** again.

Step 5: View Logs in CloudWatch

1. Scroll to the **Monitoring** tab.
2. Click **View logs in CloudWatch**
3. You'll see the `console.log()` or `print()` output with the event details.

21) Create and Manage a DynamoDB Table

=>

Step 1: Open DynamoDB Console

1. Go to <https://console.aws.amazon.com/dynamodb>
2. Click on "Create table"

Step 2: Configure the Table

- **Table name:** StudentRecords
- **Partition key (Primary Key):**
 - **Name:** StudentID
 - **Type:** String

Add a Sort key — for example, CourseName, if you want composite keys.)

- **Table settings:**
 - Leave default settings unless you want custom provisioned throughput or encryption.

Click **Create table**

Step 3: Add Items to the Table

After the table is created:

1. Go to your new table StudentRecords
2. Click on the **Items** tab
3. Click **Create item**
4. Use the JSON or form editor to enter data like:

```
{  
  "StudentID": "S001",  
  "Name": "Dipak",  
  "Age": 21,  
  "Course": "MCA"  
}
```

5. Click **Create item** to save.

Step 4: View and Manage Items

- **View Items:** Click on the **Items** tab to see all entries.
- **Edit Item:**
 - Click on an item → Actions → Edit → Update the values → Click **Save**
- **Delete Item:**
 - Select item → Actions → Delete → Confirm

Step 5: Explore Table Metrics (Optional)

1. Go to the **Monitoring** tab
2. View metrics like read/write capacity, throttling, and latency (powered by Amazon CloudWatch)

Step 6: Enable TTL or Indexes (Optional)

- **TTL (Time to Live):** Auto-delete items after a specific time.
- **Global Secondary Index (GSI) or Local Secondary Index (LSI):** Add extra indexes for querying on non-primary key attributes.

22) Create and Manage a DynamoDB Table

=>

STEP 1: Sign in to AWS

1. Go to: <https://console.aws.amazon.com/>
2. Log in with your AWS account.

STEP 2: Open DynamoDB Service

1. In the **search bar**, type DynamoDB and click on the service.
2. Click on **"Tables"** from the left menu.
3. Select your existing table (or create a new one if needed).

STEP 3: Write Data (Add Item)

1. Click on the **"Items"** tab.
2. Click **"Create item"** (blue button).
3. You'll see a form to add data. You can use the **Tree view** or switch to **JSON view**.

Example: -

```
{  
  "StudentID": "S001",  
  "Name": "Dipak",  
  "Age": 21,
```

```
"Course": "MCA"  
}
```

4. Click **“Create item”** to save it.

STEP 4: Read Data (View Items)

1. After adding data, you will see your item in the **Items list**.
2. You can:
 - Click on the item to view its full details.
 - Click **“Query”** to find an item by primary key:
 - Select your partition key (e.g., StudentID)
 - Type the key value (e.g., S001)
 - Click **“Start search”**

23) Create an Amazon RDS MySQL Database

=>

Step 1: Sign in to AWS Console

1. Go to <https://console.aws.amazon.com>
2. Log in with your credentials.

Step 2: Open RDS Dashboard

1. In the search bar, type RDS and select **Amazon RDS** service.
2. Click on **“Databases”** from the left sidebar.

Step 3: Create a Database

1. Click on **“Create database”** (orange button).

2. Under **Database creation method**, choose:
Standard Create

Step 4: Engine Options

1. Under **Engine options**, select:
MySQL
2. Choose a version (use default or latest).

Step 5: Template

Choose a use case:

- **Free tier** (recommended for learning/testing)

Step 6: Settings

1. DB instance identifier: mydbinstance
2. Master username: admin (or any name you like)
3. Master password: Choose a strong password
4. Confirm password

Step 7: DB Instance Class

For free tier:

- Select: db.t3.micro or db.t2.micro (if eligible)

Step 8: Storage

- Storage type: General Purpose (SSD)
- Allocated storage: Default (20 GiB)
- Enable storage autoscaling (optional)

Step 9: Connectivity

1. Select a VPC (default is fine)

2. Public access:
Yes (if you want to access the DB from your own computer)
3. VPC security group:
 - Choose **Create new** or **Choose existing**
 - If creating new, allow MySQL/Aurora (port 3306)

Step 10: Database Authentication

- Use password authentication (default)

Step 11: Additional Configurations

You can keep defaults or customize:

- Backup: Enable (default 7 days)
- Monitoring: Optional
- Maintenance: Default is fine

Step 12: Create Database

- Scroll down and click **“Create database”**

Step 13: Wait for RDS to Launch

- Go to **Databases**
- Wait for **Status: “Available”**
- Click on the DB instance to view its **Endpoint** and **port**

Connect to the MySQL RDS Instance

connect using:

- **MySQL Workbench**
- **Command line**

```
mysql -h <endpoint> -P 3306 -u admin -p
```

24) Backup and Restore an RDS Database

=>

Backup an RDS Database: - Amazon RDS automatically creates backups if automated backups are enabled. You can also take **manual snapshots**.

Step 1: Enable Automated Backups (if not already enabled)

1. Go to the **RDS Dashboard** → **Databases**.
2. Click on your **DB instance name**.
3. Under **Maintenance & backups**, ensure:
 - **Automated backups** is enabled.
 - **Backup retention period** is set (e.g., 7 days).

Step 2: Take a Manual Snapshot (Recommended)

1. Go to **Databases**.
2. Select your RDS instance.
3. Click **Actions** → **Take snapshot**.
4. Enter a snapshot name (e.g., mydb-backup).
5. Click **Take snapshot**.

Restore an RDS Database: - You cannot restore a snapshot to the same DB instance — AWS creates a **new DB instance** from the snapshot

Step 3: Restore from Snapshot

1. Go to the **RDS Dashboard** → **Snapshots** (left menu).
2. Choose the snapshot (automated or manual).
3. Click **Actions** → **Restore snapshot**.
4. Enter a **new DB instance identifier** (e.g., mydb-restore).

5. Choose other settings like VPC, security group, etc.
6. Click **Restore DB instance**.

Step 4: Wait for Restore

- Go to **Databases** and wait for the new instance status to show **Available**.
- Note the new **endpoint address**.

Step 5: Connect to Restored DB

Use any SQL client or CLI with:

```
mysql -h <restored-endpoint> -u admin -p
```

25) You need to deploy a Linux-based server on AWS for hosting a web application. The goal is to create an EC2 instance running on Linux, connect to it using SSH, and install Apache web server to serve a simple webpage.

=>

Part 1: Launch a Linux EC2 Instance

Step 1: Log in to AWS Console

- Go to <https://console.aws.amazon.com/> and sign in.

Step 2: Launch an EC2 Instance

1. Navigate to **EC2** → Click **Instances** → **Launch Instances**.
2. Fill in the required details:
 - **Name:** MyLinuxWebServer
 - **AMI:** Choose **Amazon Linux 2 AMI (HVM), SSD Volume Type**
 - **Instance type:** Choose t2.micro (Free Tier eligible)

- **Key pair:** Create or select an existing key pair (download .pem file if new).
- **Network settings:**
 - Allow SSH (port 22)
 - Allow HTTP (port 80)
- Click **Launch Instance**

...

Part 2: Connect to the EC2 Instance Using SSH

Step 3: Locate Public IPv4 Address

- From **EC2 Dashboard > Instances**, select your instance.
- Note the **Public IPv4 address** (e.g., 3.123.45.67).

Step 4: Open Terminal and SSH into Instance

Use your .pem file to connect:

```
chmod 400 your-key.pem
```

```
ssh -i "your-key.pem" ec2-user@<your-public-ip>
```

...

Part 3: Install Apache Web Server

Step 5: Update Packages and Install Apache

Once connected to your instance:

```
sudo yum update -y
```

```
sudo yum install httpd -y
```

Step 6: Start and Enable Apache

```
sudo systemctl start httpd
```

```
sudo systemctl enable httpd
```

Step 7: Create a Simple Web Page

```
echo "<h1>Welcome to My AWS Linux Web Server!</h1>" | sudo tee  
/var/www/html/index.html
```

...

Part 4: Test the Web Server

Step 8: Open Web Browser

```
http://<your-public-ip>
```

26) You need to store images for your web application on AWS S3. The task involves creating an S3 bucket, uploading files to the bucket, and managing the contents by deleting some objects.

=>

Part 1: Create an S3 Bucket

Step 1: Log in to AWS Console

- Go to <https://console.aws.amazon.com/>

Step 2: Open S3 Service

- From **Services**, search for and select **S3**

Step 3: Create a Bucket

1. Click **Create bucket**
2. Configure settings:
 - **Bucket name:** my-webapp-images-bucket (must be globally unique)
 - **Region:** Select your preferred AWS region

3. Uncheck **Block all public access** if images need to be accessed publicly
4. Acknowledge warning and click **Create bucket**

Part 2: Upload Files (Images)

Step 4: Upload Image Files

1. Click your bucket name
2. Click **Upload**
3. Choose image files (e.g., .jpg, .png)
4. Click **Upload**

Part 3: Manage Bucket Contents

Step 5: Delete Specific Files (Objects)

1. In the bucket, check the box next to the file(s) you want to delete
2. Click **Actions** → **Delete**
3. Confirm deletion

27) You are tasked with creating a database table to store user information, such as user ID and email address. Use DynamoDB to create a table, add some records, and query data using AWS Console.

=>

Step 1: Open DynamoDB Console

1. Log in to [AWS Console](#)
2. Navigate to **Services** → Search and select **DynamoDB**

Step 2: Create a DynamoDB Table

1. Click **Create table**

2. Fill in the table settings:
 - **Table name:** Users
 - **Partition key:** userID (Type: String)
 - Leave sort key blank (optional)
3. Leave default settings (on-demand capacity, encryption, etc.)
4. Click **Create table**

Step 3: Add Records (Items)

1. Open the newly created table Users
2. Go to the **Items** tab
3. Click **Create item**
4. Switch to **JSON view** (or use the form view) and enter:

```
{  
  "userID": "dipak001",  
  "email": "dipak001@example.com"  
}
```

5. Click **Create item**

Step 4: Query the Data

1. Go to the **Explore table items** tab
2. Select **Query** from the dropdown
3. Under **Query by primary key**:
 - Set userID → "user002"
4. Click **Run** to fetch the matching record

28) You need to set up a MySQL database on AWS to store data for a web application. The goal is to create an RDS instance with MySQL and connect to it via a MySQL client.

=>

Step 1: Sign In to AWS Management Console

- 1. Go to <https://console.aws.amazon.com/>**
- 2. Search for RDS in the services menu and click it.**

Step 2: Create an RDS MySQL Instance

- 1. Click Create database**
- 2. Choose Standard create**
- 3. Engine options:**
 - **Engine type: MySQL**
 - **Version: Choose latest stable (e.g., MySQL 8.0)**
- 4. Templates:**
 - **Choose Free tier (if eligible)**
- 5. Settings:**
 - **DB instance identifier: mydbinstance**
 - **Master username: admin**
 - **Password: Enter a secure password**
 - **Confirm password**
- 6. DB instance size:**
 - **db.t3.micro (Free tier eligible)**
- 7. Storage:**

- **Keep default (20 GB)**

Step 3: Configure Connectivity

1. Connectivity section:

- **VPC: Default (or create custom if required)**
- **Enable Public access: Yes (so you can connect)**
- **VPC security group: Create new or choose existing**

2. Leave rest of the options as default and click Create database

Step 4: Modify Security Group to Allow MySQL Access

1. Go to EC2 → Security Groups

2. Select the security group attached to the RDS instance

3. Go to Inbound rules → Edit inbound rules

4. Add a new rule:

- **Type: MySQL/Aurora**
- **Protocol: TCP**
- **Port Range: 3306**
- **Source: My IP (for better security) or Anywhere (0.0.0.0/0) if testing only**

5. Click Save rules

Step 5: Connect to the RDS Instance

1. Go back to RDS → Databases

2. Click on your DB instance

3. Copy the Endpoint (e.g., mydbinstance.xyz123.rds.amazonaws.com)

4. Use a **MySQL client** (like MySQL Workbench, DBeaver, or terminal):

Command-line example:

```
mysql -h mydbinstance.xyz123.rds.amazonaws.com -P 3306 -u admin -p
```

Enter password when prompted

Example Query (Once Connected)

...

```
CREATE DATABASE webappdb;
```

```
USE webappdb;
```

```
CREATE TABLE users (
```

```
  id INT AUTO_INCREMENT PRIMARY KEY,
```

```
  name VARCHAR(100),
```

```
  email VARCHAR(100)
```

```
);
```

```
INSERT INTO users (name, email) VALUES ('Alice', 'dipak@example.com');
```

```
SELECT * FROM users;
```

29) Develop a serverless application that automatically processes a message "I am from Lambda Function". The solution requires creating a Lambda function.

Step 1: Go to AWS Lambda Console

1. Sign in to [AWS Console](#)
2. In the **Search bar**, type **Lambda**, and click on **Lambda**

Step 2: Create a Lambda Function

1. Click **Create function**
2. Choose **Author from scratch**
3. Fill in the details:
 - **Function name:** PrintMessageFunction
 - **Runtime:** Choose Python 3.12 or Node.js (you can also choose Java or others)
 - **Execution role:** Choose Create a new role with basic Lambda permissions
4. Click **Create function**

Step 3: Add the Code

```
exports.handler = async (event) => {  
  const message = "I am from Lambda Function";  
  console.log(message);  
  return {  
    statusCode: 200,  
    body: message  
  };  
};
```

Click **Deploy** after adding the code

Step 4: Test the Function

1. Click the **Test** button
2. Choose:
 - **Event name:** TestEvent
 - **Event template:** Choose Hello World
 - Click **Create**
3. Click **Test** again

Output is: -

```
{  
  "statusCode": 200,  
  "body": "I am from Lambda Function"  
}
```

30) You need to host a static website using AWS S3. The website consists of HTML, CSS, and JavaScript files. Configure an S3 bucket for static website hosting and make it accessible over the internet.

=>

Step 1: Create an S3 Bucket

1. Go to the [S3 console](#)
2. Click "**Create bucket**"
3. Fill in:
 - **Bucket name:** e.g., my-static-website-bucket
 - **Region:** Choose your preferred region
4. **Uncheck:** "Block all public access" (important for public website)

5. Acknowledge the warning by checking the box
6. Click **"Create bucket"**

Step 2: Upload Website Files

1. Open your bucket
2. Click **"Upload"**
3. Click **"Add files"**
 - Upload your index.html, style.css, script.js, images, etc.
4. Click **Upload**

Step 3: Configure Static Website Hosting

1. In the bucket, go to the **"Properties"** tab
2. Scroll to **Static website hosting**
3. Click **Edit**
4. Select **"Enable"**
5. Choose hosting type: **"Host a static website"**
6. Set:
 - **Index document:** index.html
 - (Optional) **Error document:** error.html if available
7. Click **Save changes**

Step 4: Make Files Public

1. Go to the **"Permissions"** tab of the bucket
2. Scroll to **Bucket policy** and click **Edit**
3. Paste the following policy (replace your-bucket-name):
{

```
"Version": "2012-10-17",  
"Statement": [  
  {  
    "Sid": "PublicReadGetObject",  
    "Effect": "Allow",  
    "Principal": "*",  
    "Action": "s3:GetObject",  
    "Resource": "arn:aws:s3:::your-bucket-name/*"  
  }  
]  
}
```

4. Click **Save changes**

Step 5: Access the Website

1. Go to **Properties** → **Static website hosting**
2. Copy the "**Bucket website endpoint**" URL
 - Example: `http://your-bucket-name.s3-website-us-east-1.amazonaws.com`
3. Paste it in browser — then static website is live!

Writed by Dipak

* * *